

Deployment Strategies

CSSE6400

Richard Thomas

May 11, 2026

Branching & Deployment Strategies

- Branching Strategies
- Deployment Strategies
 - Recreate Deployment
 - Rolling Deployment
 - Blue/Green Deployment
 - Canary Deployment
 - A/B Deployment
 - Shadow Deployment
- Branching strategies help manage development activities
- Deployment strategies help manage releasing / deploying systems

Definition 0. Branching

Copying the trunk to allow separate and parallel development.

- Branches deviate from the trunk
- We will look at a few different branching strategies

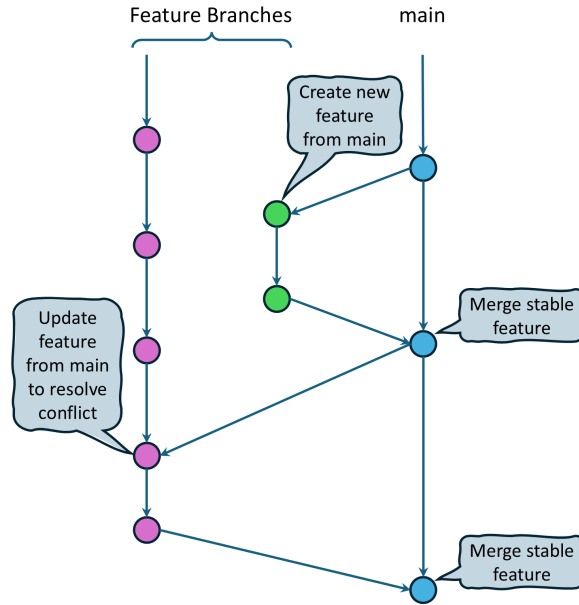
Branching Strategies

- GitHub Flow
- GitLab Flow
- Release Branches

Branching strategies supporting deployment strategies

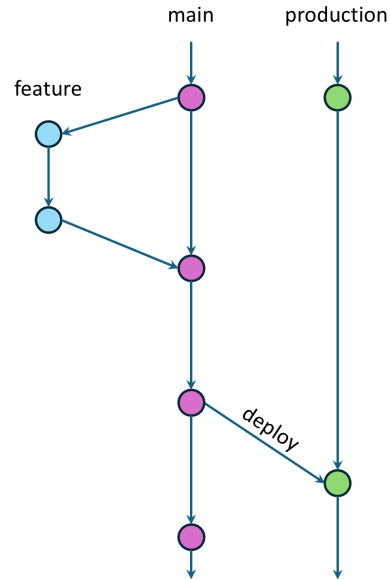
GitHub Flow ^[Haddad, 2022]

- Main is always *deployable*
- Create branch
- Make changes
- Create pull request
- Resolve issues
- Merge pull request
- Delete branch



- CSSE3200 Feature Branches are a variant of this
- Supports CI & CD
- Expects there is a *single* deployable version (e.g. cloud / web systems)

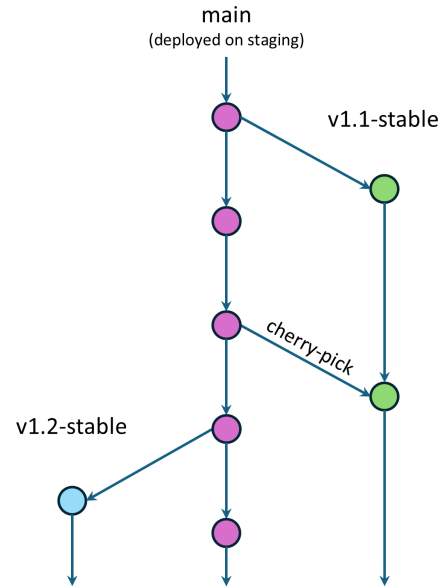
- Supports deployment *windows*
 - Merge to production
 - Deploy when allowed
- Production branch
 - Plus alpha, beta, ...
- Still have
 - Feature branches
 - Pull requests



- Deployment windows examples
 - App store approval
 - Server availability
 - Support availability

Release Branches ^[Herrera, 2022]

- Supports multiple *versions* of system
- Feature development in main
- Released versions are branches
- Bug fixes in main
 - cherry-pick into branches

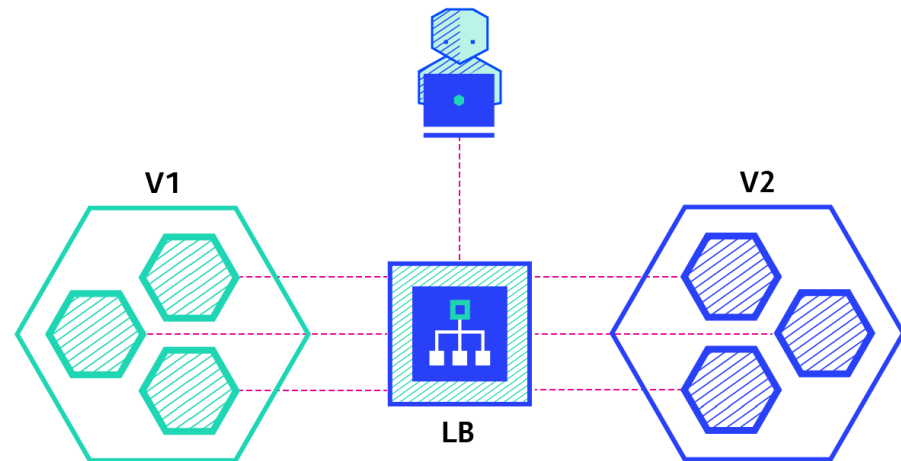


- Cherry-pick: commit is copied from one branch to another, but the branches aren't merged

Definition 0. Deployment Strategy

How a software system is made available to clients.

Recreate Deployment *[Tremel, 2017]*



- Shutdown version 1
- Deploy version 2
- Requires downtime

Recreate Deployment

Pros

- Easy
- Renewed state
 - App reinitialised
 - Persistent storage consistent with system version

Cons

- Downtime

Renewed state means app is reinitialised and db table structure is consistent with system version

Rolling Deployment *[Tremel, 2017]*

- Slowly roll out new version
- Pool of instances of **V1** behind load balancer
- Deploy an instance of **V2**
- Add **V2** instance to pool
- Remove one **V1** instance from pool
- Continue until **V2** is fully deployed, replacing **V1**

Rolling Deployment

Pros

- Fairly easy
- Slow release of new version
 - Observe issues
 - Rollback
- Stateful instances can finish gracefully
 - Instance is killed when inactive

Cons

- Time
- Support multiple APIs
- Support different versions of persistent data structure
- No control over traffic to different versions

Blue-Green Deployment *[Tremel, 2017]*

- **V2** deployed alongside **V1**, including same number of instances
- **V2** tested in production environment
- Load balancer switched to use **V2** instances
- Shutdown **V1** instances

Blue-Green Deployment

Pros

- Instant release of new version
- Fast rollback if necessary
- Only one version 'live' at any time
 - No versioning conflicts

Cons

- Expensive
 - Double the infrastructure
- Stateful instance version switch difficult
 - Can't kill instance in middle of a transaction

Canary Deployment *[Tremel, 2017]*

- Gradually shift traffic from **V1** to **V2**
- Traffic usually split by percent (e.g. 90/10, 80/20, ...)
- Allows a trial deployment to see what happens

Canary Deployment

Pros

- New version released to subset of users
- Can monitor performance and error rates
- Easy and fast rollback

Cons

- Slow
- Implies poor testing

Commonly used to see if something works or will fail in production

A/B Deployment *[Tremel, 2017]*

- Actually it's A/B Testing
- Both versions are deployed and usage evaluated
 - Usually via analytics
- Long Term: Deploy version that has best usage result

A/B Deployment

Pros

- Multiple versions run in parallel
- Full control over traffic distribution

Cons

- Needs intelligent load balancer
- Debugging a version is difficult
 - Need good logs & tools

Requires sophisticated infrastructure and analytics to do well

Shadow Deployment *[Tremel, 2017]*

- **V2** deployed alongside **V1**
- All traffic is sent to **V1** *and* **V2**
- Tests **V2** ability to handle production load
- Doesn't impact on production traffic or user experience
- **V2** rolled out when it demonstrates it is stable
- Need to manage interactions with external services (e.g. payment gateway)
 - When customer checks out their shopping cart, you don't want to send *two* payment requests from **V1** *and* **V2**
 - *Mock* external services
- Persistent data from **V1** (production data) needs to be copied to **V2** when it's deployed as production
 - With any data migration

Shadow Deployment

Pros

- Performance testing with production traffic
- No impact on users

Cons

- Expensive
 - Double the infrastructure
- Complex to setup
 - Need mocks for external services

Performance testing may give false confidence

- It's not user testing

Deployment Strategy Options

- Staging or beta testing
 - Recreate or Rolling
- Production (Live)
 - Rolling or Blue/Green
- Uncertain of system stability
 - Canary
- Evaluation
 - A/B or Shadow

No perfect deployment strategy

- There are always tradeoffs

Deployment Considerations *[Tremel, 2017]*

Strategy	ZERO DOWNTIME	REAL TRAFFIC TESTING	TARGETED USERS	CLOUD COST	ROLLBACK DURATION	NEGATIVE IMPACT ON USER	COMPLEXITY OF SETUP
RECREATE version A is terminated then version B is rolled out	✗	✗	✗	■□□	■ ■ ■	■ ■ ■	□ □ □
RAMPED version B is slowly rolled out and replacing version A	✓	✗	✗	■□□	■ ■ □	■ □ □	■ □ □
BLUE/GREEN version B is released alongside version A, then the traffic is switched to version B	✓	✗	✗	■ ■ ■	□ □ □	■ ■ □	■ ■ □
CANARY version B is released to a subset of users, then proceed to a full rollout	✓	✓	✗	■ ■ □	■ □ □	■ □ □	■ ■ □
A/B TESTING version B is released to a subset of users under specific condition	✓	✓	✓	■ ■ □	■ □ □	■ □ □	■ ■ ■
SHADOW version B receives real world traffic alongside version A and doesn't impact the response	✓	✓	✗	■ ■ ■	□ □ □	□ □ □	■ ■ ■

- Modification of original diagram at <https://thenewstack.io/deployment-strategies/>
- Cloud Cost for Canary & A/B *increased* to *medium*
 - Rollback Duration for Ramped *decreased* to *medium*

References

- [Haddad, 2022] Haddad, R. (2022).
What are the best git branching strategies.
<https://www.abtasty.com/blog/git-branching-strategies/>.
- [Herrera, 2022] Herrera, M. (2022).
Git branching strategies in 2022.
[https://faun.dev/co/stories/manuelherrera/
git-branching-strategies-in-2022/](https://faun.dev/co/stories/manuelherrera/git-branching-strategies-in-2022/).
- [Saavedra, 2024] Saavedra, C. (2024).
Combine gitlab flow and gitlab duo for a workflow powerhouse.
<https://about.gitlab.com/blog/gitlab-flow-duo/>.
- [Tremel, 2017] Tremel, E. (2017).
Six strategies for application deployment.
<https://thenewstack.io/deployment-strategies/>.